

Problem Setting: Bird Species Identification from Passive Acoustic Monitoring

Overview

This project is based on the BirdCLEF+ 2026 Kaggle competition, which focuses on automated biodiversity monitoring in the Pantanal wetlands of Brazil using passive acoustic monitoring (PAM). A growing network of 1,000 acoustic recorders is deployed across the Pantanal, producing continuous multi-species soundscapes that are too large to review manually. The goal is to develop a machine learning model that can automatically identify bird species from these recordings, supporting evidence-based conservation decisions.

Formal Problem Setting

Let $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$ be a fixed set of K bird species (classes), selected as a subset of the full competition taxonomy. Given an audio recording $\mathbf{x} \in \mathbb{R}^T$ of fixed duration T seconds, the goal is to predict a binary label vector $\mathbf{y} \in \{0, 1\}^K$, where $y_k = 1$ indicates that species c_k is vocalizing in the recording.

This is a **multi-label audio classification** task.

Scope Reduction

The full competition dataset contains hundreds of species with highly imbalanced class distributions. To keep the project computationally tractable, we restrict to the **top K most frequent species** by number of training examples, where K is determined empirically after inspecting the dataset’s frequency distribution. Species with fewer than 100 labeled recordings are excluded. We expect K to fall in the range of 25–35, balancing the richness of the multi-label problem against computational constraints on free-tier compute (Kaggle Notebooks / Google Colab).

Input Representation

Raw waveforms are converted to **log-scaled mel-spectrograms** of shape $(F \times T')$, where $F = 128$ mel frequency bins and T' is the number of time frames corresponding to a fixed 5-second audio clip. Each recording is chunked into non-overlapping 5-second windows at inference time; predicted probabilities across windows are aggregated via max-pooling to produce a single clip-level prediction.

This representation transforms the audio classification problem into a 2D image classification problem, where the x-axis encodes time, the y-axis encodes

frequency, and pixel intensity encodes energy. This makes it directly amenable to vision-based deep learning architectures.

Proposed Architecture: CNN-Transformer Hybrid

Motivation

A standard Vision Transformer (ViT) divides the spectrogram into fixed-size patches and linearly projects each patch into a token embedding before passing them to a Transformer encoder. While effective, this approach has a key limitation for acoustic data: **raw patch projections have no inductive bias toward local time-frequency patterns**. Bird calls are characterized by highly structured local features (short chirps, harmonic stacks, frequency trills) whose precise shape is diagnostically important for species identification. A patch boundary that falls mid-chirp may fragment these features before the model has any chance to detect them.

At the same time, a pure CNN lacks the ability to model **long-range temporal dependencies** across the clip. A call segment at second 1 and a response at second 4 may together be diagnostic of a species, but a CNN would require very deep stacking of layers to capture such context.

A further practical motivation for the hybrid design concerns **sequence length**. A raw mel-spectrogram of shape 128×313 (128 frequency bins, 313 time frames for a 5-second clip at a typical hop length) would produce over 40,000 tokens if flattened directly, making self-attention computationally very hard. The CNN front-end, through successive strided convolutions, downsamples the spatial dimensions significantly before the Transformer ever sees the input. With 3–4 CNN blocks using stride 2, the resulting sequence length is on the order of 100–200 tokens, well within the range where standard self-attention is efficient. The CNN thus solves the sequence length problem naturally as a side effect of downsampling.

Architecture

We propose a **CNN-Transformer hybrid** that combines the strengths of both architectures:

1. **CNN front-end**: A lightweight convolutional backbone (e.g. 3–4 ResNet-style blocks) processes the full mel-spectrogram and produces a feature map $\mathbf{F} \in \mathbb{R}^{C \times H' \times W'}$. The CNN extracts local acoustic patterns: edges, harmonics, onset contours at multiple scales before any global reasoning takes place.
2. **Positional encoding**: Before entering the Transformer, each token is summed with a **2D learnable positional embedding**, one learned vector per spatial position (h, w) in the feature map grid. This is preferable

over 1D sinusoidal encodings because the tokens retain a 2D spatial structure (time $\times$ frequency) after the CNN, and a 1D encoding would lose the distinction between the two axes.

3. **Transformer encoder:** The positionally-encoded token sequence $\mathbf{Z} \in \mathbb{R}^{(H' \cdot W') \times C}$, prepended with a learnable **[CLS]** token, is passed through a standard Transformer encoder with multi-head self-attention. The number of layers and attention heads are treated as hyperparameters. This allows the model to reason about relationships between distant time-frequency regions, capturing global call structure and temporal context across the full clip.
4. **Classification head:** The output representation at the **[CLS]** position is passed through an **MLP head** which has two linear layers with a GELU activation and dropout in between, producing K logits, followed by a sigmoid activation for independent multi-label prediction per species.

This design is directly motivated by the two-level structure of bird vocalizations: local call morphology (handled by the CNN) and global temporal context (handled by the Transformer).

Baselines

Model	Description
Simple baseline	MFCC feature extraction (mean + standard deviation over time) + Random Forest classifier, trained independently per class in a one-vs-rest setup. No deep learning, no spectrogram.
ML baseline	Standard ResNet-18 trained directly on mel-spectrograms, without any attention mechanism. This isolates the contribution of the Transformer component by comparing against a pure CNN of similar capacity.
ViT baseline	A standard Vision Transformer applied to raw spectrogram patches, without a CNN front-end. This isolates the contribution of the CNN front-end by comparing against a pure Transformer.

The three-way comparison (Random Forest $\rightarrow$ ResNet $\rightarrow$ ViT $\rightarrow$ CNN-

Transformer) tells a clear and progressive story: each step adds a motivated component, and results should reflect the value of each addition.

Evaluation Protocol

Data Split

- **Training set:** official competition training data, restricted to the top K selected species.
- **Validation set:** The validation set is set to 15% of the training data by default. This proportion may be adjusted downward if per-class sample counts after filtering are too low to preserve sufficient training examples, as determined by inspecting the dataset distribution.
- **Test set:** official competition test set. If this is not available or sufficiently labeled, a stratified holdout from the training data will be used instead, with the exact proportion determined after inspecting the dataset size.

All models are evaluated on the **same** validation and test splits to ensure results are directly comparable.

Metric

The primary metric is **macro-averaged ROC-AUC** across all K species, consistent with the official Kaggle competition metric. This is the natural choice for multi-label classification with class imbalance, as it evaluates ranking performance per class independently of threshold selection and weights all species equally regardless of frequency.

As a secondary metric, we report **macro-averaged F1-score**, where the classification threshold is selected per class on the validation set by maximizing per-class F1, rather than using a fixed default.

Hyperparameter Tuning

The following hyperparameters will be tuned on the validation set via random search over a small candidate grid:

- **CNN-Transformer:** number of CNN blocks, CNN output channels, number of Transformer layers, number of attention heads, model dimension d_{model} , dropout rate, learning rate, patch size after CNN.
- **ViT baseline:** patch size, number of Transformer layers, learning rate, dropout rate.
- **ResNet baseline:** depth (ResNet-18 vs. ResNet-34), learning rate, augmentation strategy.
- **Random Forest:** number of trees, max depth, MFCC feature dimension.

All models use the same data augmentation pipeline (SpecAugment time and frequency masking) to ensure a fair comparison.

Repository Structure

```
aml2026-group10/  
  PROBLEMSETTING.md  
  data/  
    preprocessing/      # mel-spectrogram extraction, chunking, augmentation  
  models/  
    cnn_transformer.py  # proposed hybrid architecture  
    vit_baseline.py     # ViT baseline  
    cnn_baseline.py     # ResNet baseline  
    rf_baseline.py      # MFCC + Random Forest baseline  
  train.py  
  evaluate.py  
  slides/
```